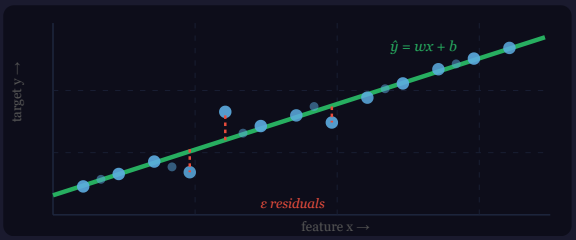


A VISUAL INTERVIEW REFERENCE

NEURAL CHRONICLES

THE SIGNAL CODEX

From linear regression to transfer learning — the mathematics, intuition, and working TensorFlow code behind modern neural networks. Every chapter ends with an interview checklist.

Linear Regression · Logistic Regression · Neural Networks · Backprop · CNNs · LSTMs · Transfer Learning · The Complete Metrics Story

TENSORFLOW EDITION · INTERVIEW-READY REFERENCE

PROLOGUE

Three Data Types That Break Classical ML

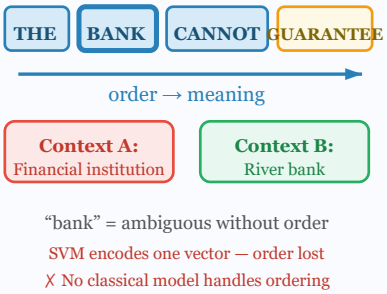
KNN, Decision Trees, SVM, Random Forest — these algorithms share one fundamental assumption: features are independent scalars in a flat vector. Three data types violate this so severely that no amount of tuning helps. Understanding *why* is the setup for every deep learning interview question that follows.

FILE A — IMAGE DATA

6 × 4 pixel grid (spatial)
position encodes meaning

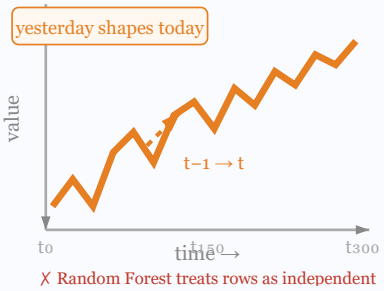
KNN measures Euclidean distance across 50,000 pixel dimensions — meaningless. Trees split on pixel$_{14832}$ with no spatial context. **Position is the signal; flattening destroys it.**

FILE B — SEQUENCE DATA



Meaning depends on position and context. Decision trees split on *feature values*, never on word order. SVM treats the whole row as one independent vector.

FILE C — TIME SERIES



Day 499 depends on day 498. The Knight's hyperplane has no memory. Gen. Bagging draws random bootstraps — no temporal order enforced.

Why Every Classic Tool Fails — At a Glance

Model	Image (spatial)	Text (ordered)	Time Series	Root Cause
KNN	✗ Distance in 50k dims meaningless	✗ Cosine ignores order	✗ No temporal structure	Flat feature vectors only
Decision Tree	✗ Splits on pixel$_{14832}$	✗ No sequence splits	✗ Rows assumed independent	Axis-aligned splits only
SVM / Ensemble	✗ Quadratic cost in high dims	✗ One vector per sentence	✗ No recurrence	Static feature vector assumption
Solution	**CNN (spatial filters)**	**LSTM (sequential memory)**	**GRU (recurrent state)**	**Architecture that fits data structure**

Each case demands architecture that matches the data structure: spatial filters (CNNs) for images, sequential memory (LSTMs/GRUs) for text and time series. The prerequisite to all of it: understanding the simplest possible predictive model — the one Legendre formalised in 1805.

GUIDE OVERVIEW

What's In This Guide — And What You'll Be Asked

Fourteen chapters across two sections. Ch.1–9 cover deep learning foundations, from linear regression through RNNs. Ch.10–14 cover classical ML and unsupervised learning — Decision Trees, SVM, XGBoost, clustering, dimensionality reduction, and their evaluation metrics. Every chapter ends with a three-column interview checklist.

Chapter Map & Interview Topic Index

#	Topic	Key Concepts	Common Interview Questions
1	**Linear Regression**	$\hat{y} = wx + b$ · MSE · MAE · R^2 · Gradient Descent · Learning Rate	MSE vs MAE? What does $R^2 < 0$ mean? How does `Dense(1)` relate to linear regression?
2	**Logistic Regression**	Sigmoid · Binary Cross-Entropy · Log-loss · Decision Boundary · AUC	Why not MSE for classification? What is the accuracy trap? When does AUC beat accuracy?
3	**The XOR Problem**	Linear separability · Why single-layer perceptrons fail · Universal approximation theorem	What is the XOR problem? What caused the 1969 AI winter? Wide vs deep networks?
4	**Neural Networks**	Forward pass · ReLU / Sigmoid / Softmax · Weight initialisation · Output layer design	Which activation on the output for multi-class? Why ReLU over Sigmoid in hidden layers?
5	**Backprop & Optimisers**	Chain rule · SGD · Momentum · RMSProp · Adam · LR schedules	Explain backprop in one sentence. Why does Adam outperform SGD in practice? What is gradient clipping?
6	**Regularisation**	L1 · L2 · Dropout · Batch Normalisation · Early Stopping	L1 vs L2: which produces sparse weights? What does dropout do at inference? Vanishing gradient?
7	**CNNs**	Convolution · Feature maps · Pooling · Receptive field · Skip connections	Why can't dense layers handle images? What is parameter sharing? How do residual connections help?
8	**RNNs / LSTMs / GRUs**	Hidden state · Vanishing gradient · LSTM gates · GRU · Bidirectional	Why do vanilla RNNs fail on long sequences? What are the three LSTM gates? GRU vs LSTM?
9	**Metrics Deep Dive**	Precision · Recall · F1 · ROC-AUC · PR-AUC · BLEU · Perplexity	Precision vs Recall trade-off. When is PR-AUC better than ROC-AUC? What does a BLEU score measure?
10	**Classical Classifiers**	Gini · Entropy · Information Gain · max_depth · KNN · k selection · Curse of dimensionality	Gini vs entropy? Why scale before KNN? Decision tree vs logistic regression — when does each win?
11	**SVM & Ensembles**	Margin · Support vectors · C · Kernel trick · RBF γ · Bagging · Boosting · XGBoost η	What is the kernel trick? C in SVM? Bagging vs boosting? Why XGBoost over Random Forest?
12	**Clustering**	K-Means · Inertia · Elbow method · DBSCAN · ε · min_samples · HDBSCAN · Feature engineering	K-Means vs DBSCAN failure modes? Core/border/noise? How to choose k? HDBSCAN vs DBSCAN?
13	**Dimensionality Reduction**	PCA · Explained variance · t-SNE · Perplexity · UMAP · n_neighbors · min_dist	t-SNE vs UMAP? What does perplexity control? Why not use t-SNE embeddings as features?
14	**Unsupervised Metrics**	Silhouette · Davies-Bouldin · ARI · Explained variance ratio · Reconstruction error · Neighbourhood preservation	Silhouette ≈ 0 means? When is ARI not useful? Trustworthiness vs continuity in UMAP/t-SNE?

🔒 The Hyperparameter Pattern

Every chapter introduces a new complexity dial. The same question repeats: *how much do I trust the training signal?*

Ch.1 — **learning rate** α (step size)

Ch.2 — **decision threshold** (precision/recall trade-off)

Ch.6 — **dropout rate** · **L2 λ** · **patience**

Ch.10 — **max_depth** (DT) · **k** (KNN)

Ch.11 — **C** · **γ** (SVM) · **η** · **n_estimators** (XGBoost)

Ch.12 — **ε** · **min_samples** (DBSCAN)

Ch.13 — **perplexity** (t-SNE) · **n_neighbors** (UMAP)

How to use this guide the night before an interview:

General ML loop → Ch.1–5

Computer vision role → add Ch.7

NLP / sequence role → add Ch.8

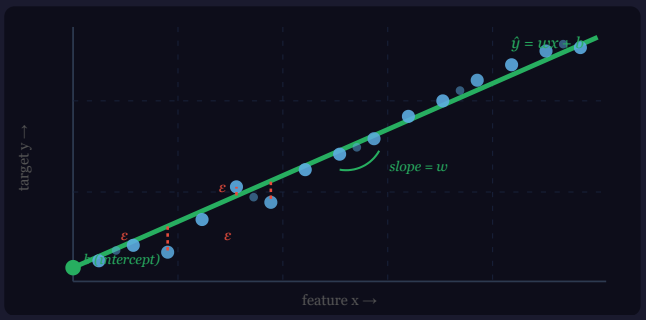
ML engineering / evals → Ch.9 + Ch.14

Classical ML / data science → Ch.10–11

Clustering & unsupervised → Ch.12

Dimensionality reduction → Ch.13

Each chapter's final page = three-column interview checklist.



CHAPTER 1

Linear Regression

LINEAR REGRESSION

The foundation of all supervised learning. Every neuron in every neural network performs this exact operation. Understanding it fully — the math, the loss, the training loop — means understanding the forward pass of every model you will ever train.

least squares · residuals · MSE · MAE · RMSE · R^2 · gradient descent · learning rate

The Model: A Weighted Sum

Historical context: In 1805, Adrien-Marie Legendre was tracking comet trajectories in Paris. He published *moindres carrés* — least squares — as a method for finding the line that was *least wrong* across all observations simultaneously. He did not know he was doing machine learning. The core criterion (minimise the sum of squared residuals) is still the default loss for regression 220 years later.

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b \;=\; \mathbf{w}^\top \mathbf{x} + b$$

What Each Term Does

- $\hat{y}$ — the *prediction*. Not truth — the model's best estimate of it.
- w_i — weight for feature i. Interpretation: each additional unit of x_i changes the prediction by w_i. These are *learned from data*.
- b — bias. The predicted value when all features are zero. Also learned.
- n — number of features. 1 for simple regression, millions for embedding layers.

1805 — The History That Built ML

Legendre published *Mémoire sur la méthode des moindres quarrés*. Gauss claimed to have used it since 1795 (unverified). The priority dispute created the first open-source argument in mathematics.

1805 — Legendre: least squares published

1880s — Galton: "regression to the mean"

1950s — Gradient descent first computed digitally

Today — Still the innermost operation of every neuron

🔒 **Interview critical:** A neural network's `Dense(1)` layer with no activation function *is* linear regression. Identical mathematical operations. When you understand linear regression, you understand the computation inside every neuron in every deep learning model you will build.

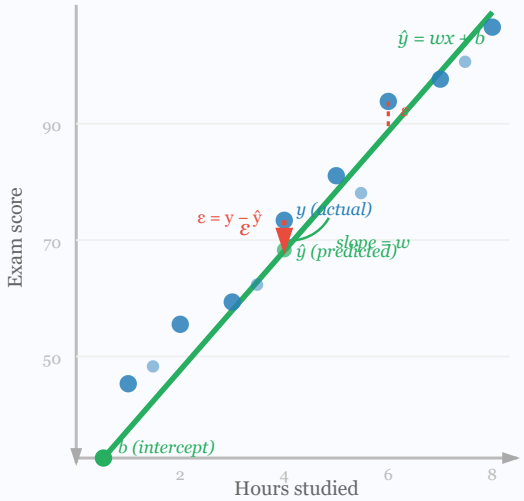


Fig 1.1 — ŷ = wx + b minimises total squared residuals ε = y − ŷ. Positive ε: model underestimates. Negative ε: model overestimates. Green dot = ŷ (on line). Blue dot = y (actual data point).

Cost Functions & Regression Metrics — Born in 1805

Why We Square the Residuals — MSE

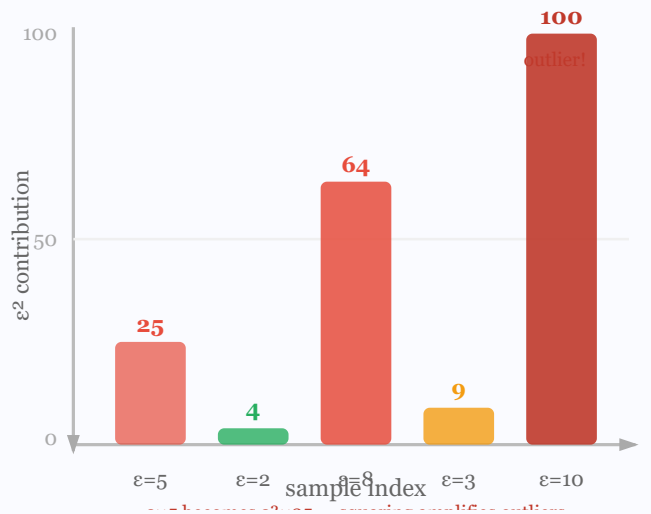


Fig 1.2 — MSE = ∑ε²/n. An outlier with ε=10 contributes 100 to cost — 4× more than ε=5 which contributes 25. ***Squaring disproportionately penalises large errors.***

MSE vs MAE — Outlier Sensitivity

Fig 1.3 — Same data. MSE heavily pulls the fit toward the outlier (squared penalty). MAE is more robust (linear penalty). Choose based on whether outliers should dominate the optimisation signal.

The Four Regression Metrics — A Complete Reference

Metric	Formula	Units	Best for	Weakness
MSE	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$	y² (e.g. £²)	Training loss — smooth gradient, differentiable everywhere	Large errors dominate; hard to interpret squared units
RMSE	$\sqrt{\text{MSE}}$	Same as y (£)	Reporting accuracy to stakeholders — same scale as target	Still outlier-sensitive (square root of MSE)
MAE	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$	Same as y	Robust reporting; outliers are noise not signal	Non-differentiable at 0; less smooth for gradient descent
R²	$1 - \frac{SS_{res}}{SS_{tot}}$	Unitless [0,1]	Proportion of variance explained vs baseline (mean predictor)	Can be negative if model is worse than predicting mean

🔒 **R² demystified:** R² = 1 means all variance explained. R² = 0 means no better than always predicting the mean ȳ. R² < 0 means you have a bug, wrong features, or severe overfit. Ask: "Is my model better than just predicting the average?"

⚠ **When to use which:**
Training → MSE (smooth gradient).
Reporting → RMSE (same unit as prediction).
Outliers are noise → MAE.
Model comparison → R².

Training: Gradient Descent — Rolling Down the Loss Surface

The Loss Landscape — Convex for Linear Regression

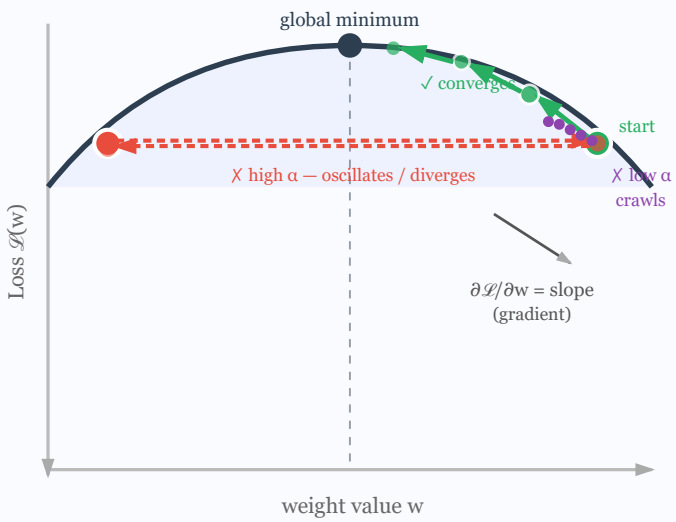


Fig 1.4 — For linear regression the loss surface is a perfect bowl (convex). Gradient descent takes steps proportional to the slope at the current position. Step size = learning rate α.

The Weight Update Rule

$$w \leftarrow w - \alpha \cdot \frac{\partial \mathcal{L}}{\partial w}$$

- **α** (learning rate) — the step size. The new Dial Thread entry.
- **∂ℒ/∂w** — gradient: which direction is uphill? Go opposite.
- One update per weight, per training batch, every epoch.

Learning Rate Effect on Training Loss

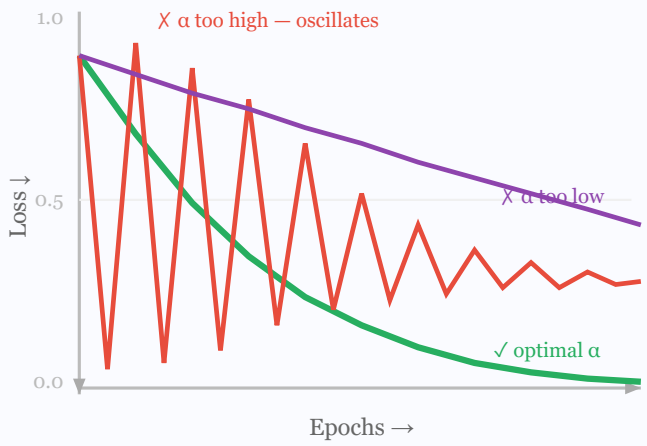


Fig 1.5 — Three learning rates on identical architecture. Optimal α converges smoothly. Too high: zigzags indefinitely. Too low: learns, but takes far too long to converge.

⚙ THE DIAL — BOOK II OPENS

Book I: K → depth → n_estimators → C

α (learning rate) is the Neural Realm's first dial.

α too high	α optimal	α too low
Overshoot & diverge	Smooth convergence	Crawls & stalls

TensorFlow Implementation — Linear Regression as a Single Neuron

Key insight: A linear regression model *is* a 1-layer neural network with no activation function. The mathematical operations are identical. `Dense(1)` with no activation = the 1805 least-squares formula running on a GPU. This is why interviewers call it "the building block of deep learning."

```
import tensorflow as tf
import numpy as np

# ── Data: y = 3x + 7 + Gaussian noise ──────────────────────
np.random.seed(42)
X = np.random.rand(500, 1).astype(np.float32)
y = (3 * X + 7 + 0.2 * np.random.randn(500, 1)).astype(np.float32)

X_train, X_test = X[:400], X[400:]
y_train, y_test = y[:400], y[400:]

# ── Model: Dense(1) with NO activation = linear regression ──
model = tf.keras.Sequential([
    tf.keras.layers.Dense(
        units=1,          # one output neuron
        input_shape=(1,) # one feature
        # no activation -> output = w*x + b directly
    )
])

# ── Compile: Legendre's 1805 criterion ──────────────────────
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
    loss='mean_squared_error',    # MSE: the original LS criterion
    metrics=['mae']               # also track MAE
)

# ── Train ───────────────────────────────────────────────────
history = model.fit(
    X_train, y_train,
    epochs=200,
    validation_split=0.2,
    verbose=0
)

# ── Evaluate all four metrics ───────────────────────────────
mse, mae = model.evaluate(X_test, y_test, verbose=0)
rmse = np.sqrt(mse)

y_pred = model.predict(X_test, verbose=0)
ss_res = np.sum((y_test - y_pred) ** 2)
ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
r2 = 1 - ss_res / ss_tot

print(f"MSE:  {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"MAE:  {mae:.4f}")
print(f"R²:   {r2:.4f}")   # expect ~0.99

# ── Inspect learned weights ─────────────────────────────────
w, b = model.layers[0].get_weights()
print(f"Learned w = {w[0][0]:.3f}  (true: 3.0)")
print(f"Learned b = {b[0]:.3f}  (true: 7.0)")
```

🔒 **The Dense(1) insight:**
Every neuron in every layer of every deep network does the same thing as this model: computes $z = \mathbf{w}^\top \mathbf{x} + b$. The only addition is an activation function $a = \sigma(z)$ after the sum. Understanding linear regression = understanding the forward pass of every neural network you will ever train.

What the Training Loop Does (Step by Step)

1. **Forward pass:** compute $\hat{y} = wx + b$ for each sample in batch
2. **Loss:** MSE = $\sum(y - \hat{y})^2/n$
3. **Backward pass:** compute $\partial L/\partial w$ and $\partial L/\partial b$
4. **Update:** $w \leftarrow w - \alpha \cdot \partial L/\partial w$
5. **Repeat** for all batches, all epochs

Practical hyperparameter starting points:
Use **Adam (lr=1e-3)** for neural nets (Ch.5). SGD works here because linear regression's loss surface is convex. For non-convex surfaces, Adam almost always outperforms plain SGD.

Chapter 1 — Core Interview Checklist

The Model

- $\hat{y} = w \cdot x + b$: weighted sum of features

The Metrics

- MSE = $\sum\varepsilon^2/n$ — training loss, outlier-

The Training

- Gradient descent: $w \leftarrow w - \alpha \cdot \partial L/\partial w$

Linear Regression — What Interviewers Actually Ask

Q: MSE vs MAE — when do you choose each?

MSE squares errors: a 10× error costs 100× as much. Use when outliers represent real signal you want corrected aggressively, and when you need smooth gradients everywhere. **MAE** penalises linearly — robust to outliers. Use when outliers are noise. **Huber loss** combines both: MSE below threshold δ, MAE above it. Both can identify the same optimal model; the choice affects which model wins under which data distribution.

Q: What does $R^2 < 0$ mean?

$R^2 = 1 - SS_{res}/SS_{tot}$. Negative means the model predicts worse than predicting ŷ = &ybar; for every sample. Not a bug — a sign the model is fundamentally misapplied: wrong distribution, severe train/test mismatch, or wrong model type entirely. Also: use **Adjusted R^2** when comparing models with different feature counts — plain R^2 always inflates with more features.

Q: Why report RMSE instead of MSE?

MSE is in y^2 units. For a housing model where y is in dollars, MSE = 40,000 is uninterpretable; RMSE = √40,000 ≈ 200 means predictions are off by ~$200. Both reach the same optimal weights — RMSE is purely a communication choice. Convention: MSE as training loss (simpler derivatives), RMSE and MAE in evaluation reports and dashboards.

Q: Batch, step, epoch — define precisely.

Batch: sample subset processed before one weight update. **Step:** one forward + backward pass on one batch → one gradient update. **Epoch:** one full pass through the entire training set. With 400 samples and `batch_size=32`: 13 steps/epoch. 200 epochs → ~2,600 total updates. Increasing batch size reduces noise but costs more memory per step.

Q: Is `Dense(1)` with no activation linear regression?

Yes — mathematically identical. Dense(1) computes z = w·x + b; without activation, output = z = ŷ. TF learns w and b by minimising MSE via gradient descent, which converges to the same solution as the closed-form normal equation

⚠ Common Interview Traps

- **Comparing R^2 across models with different feature counts:** R^2 always increases with more features. Use Adjusted R^2, which penalises unnecessary predictors.
- **Reporting raw MSE as the error:** MSE is in y^2 units. Always convert to RMSE for a human-readable error magnitude.
- **Skipping feature scaling:** gradient descent converges much faster when features share similar magnitudes. Standardise (z-score: μ=0, σ=1). Affects convergence speed only — not the optimal solution.
- **Multicollinearity:** correlated features still produce valid predictions, but individual weights become unstable and uninterpretable. Detect with VIF. Fix with L2 regularisation (Ch.6).
- **Treating linear regression as unrelated to neural networks:** `Dense(1)` + no activation IS linear regression. This bridge is tested explicitly in ML interviews.

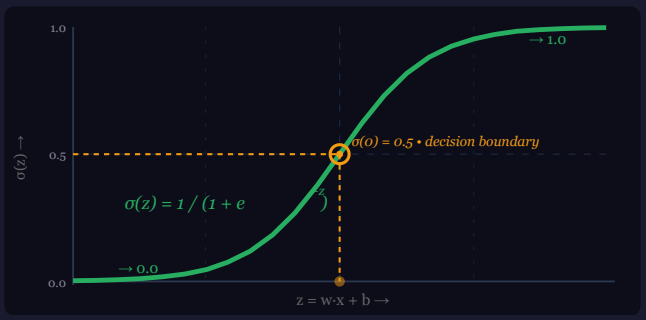
Assumptions of Linear Regression

Only **linearity** affects prediction quality. The rest matter for statistical *inference* (p-values, confidence intervals), not prediction:

1. **Linearity** — E[y|x] is actually linear in x. Violating this makes predictions wrong regardless of sample size.
2. **Independence** — no autocorrelation in residuals.
3. **Homoscedasticity** — Var(ε|x) constant, not funnel-shaped across x.
4. **No severe multicollinearity** — for stable, interpretable weights.
5. **Normal residuals** — only required for p-values & confidence intervals.

🔒 The Bridge Fact Interviewers Test

"What is the simplest possible neural network?" — `Dense(1)` with no activation, trained on MSE. That is linear regression. Every deeper network adds exactly two things: more layers and an activation function g(z) after each z = w·x + b. The entire edifice of deep learning extends from this one operation.



CHAPTER 2

Logistic Regression

LOGISTIC REGRESSION

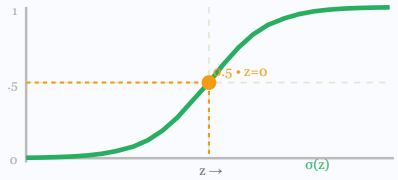
One activation function added to linear regression produces the most widely deployed binary classifier in industry. The sigmoid squashes any real-valued score into a probability — and gradient descent finds the decision boundary that separates two classes.

sigmoid · log-odds · binary cross-entropy · decision threshold · precision · recall · AUC-ROC

The Sigmoid: From Linear Score to Probability

Connection to Ch.1: Linear regression outputs ŷ = w·x + b — a continuous number, potentially negative or greater than 1. For binary classification we need P(y=1|x) ∈ [0,1]. The fix is one function wrapping the output: **sigmoid**. Every weight, every gradient update, every training step is otherwise identical.

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b) \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$



Sigmoid maps $z \in (-\infty, +\infty)$ to $\hat{y} \in (0, 1)$. Decision boundary at $z = 0$.

Term Breakdown

- $z = \mathbf{w}^\top \mathbf{x} + b$ — the *logit*: same weighted sum as Ch.1. Unbounded real number.
- $\sigma(z)$ — sigmoid squashes $z \in (-\infty, +\infty)$ into $\hat{y} \in (0, 1)$. Interpreted as P(y=1 | x).
- $\sigma(0) = 0.5$ — the decision boundary. Above → predict Class 1. Below → Class 0.
- **Decision boundary** is where $z = \mathbf{w}^\top \mathbf{x} + b = 0$ — a *hyperplane*. Logistic regression is a **linear classifier**.

Log-Odds Interpretation

Inverting the sigmoid gives a linear model in the log-odds (logit):

$$\ln \frac{\hat{y}}{1 - \hat{y}} = \mathbf{w}^\top \mathbf{x} + b$$

Each weight w_i changes the log-odds by w_i per unit increase in x_i, which is a multiplicative change of e^{w_i} in the odds ratio. The decision boundary is linear in feature space regardless of what the sigmoid does to the output.

Ch.1 vs Ch.2 — What Changed

	Linear Regression	Logistic Regression
Output	ŷ = z	ŷ = σ(z)
Range	(−∞, +∞)	(0, 1)
Loss	MSE = ∑(y−ŷ)²	BCE = −∑y·logŷ
Task	Regression	Binary classification
Weights	Same w, b — same gradient descent	

Activation derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Maximum 0.25 at $z = 0$. Shrinks toward zero at both ends — why sigmoid is replaced by ReLU in hidden layers (Ch.4).

Why not MSE for binary classification?

- **Non-convex loss surface:** MSE + sigmoid produces local minima. Gradient descent is not guaranteed to find the global optimum.
- **Gradient vanishes at saturation:** $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \approx 0$ when $\hat{y}$ is near 0 or 1. MSE passes this tiny derivative through, making weight updates nearly zero.
- **Fix:** binary cross-entropy is convex with sigmoid and its gradient $\partial L / \partial z = \hat{y} - y$ has no vanishing term.

Binary Cross-Entropy & Evaluation Metrics

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\left[\, y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

Why This Loss Works

- $y = 1, \hat{y} \to 1$: $-\log(1) = 0$ — zero loss. Perfect.
- $y = 1, \hat{y} \to 0$: $-\log(0) \to +\infty$ — severe penalty for confident misses.
- $y = 0, \hat{y} \to 0$: $-\log(1 - 0) = 0$ — zero loss. Correct.
- BCE + sigmoid = **convex** surface. Single global minimum. Gradient: $\hat{y} - y$.

The Accuracy Paradox

Dataset: 95% class-0 samples. Model predicts "always class 0": **95% accuracy, zero predictive value**. Accuracy is misleading whenever classes are imbalanced and the minority class is what matters.

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

- **Precision:** of all predicted positives, what fraction is correct? Penalises false alarms.
- **Recall:** of all actual positives, what fraction was found? Penalises missed cases.
- **F1:** harmonic mean — punishes extreme P/R imbalance. Use for imbalanced data.

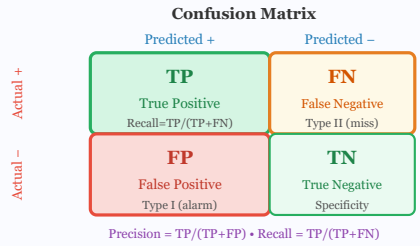


Fig 2.2 — Standard confusion matrix. Rows = actual class, columns = predicted class.

AUC-ROC: Threshold-Independent Evaluation

The **ROC curve** plots True Positive Rate (Recall) vs False Positive Rate at every threshold. **AUC** = area under that curve.

- AUC = 0.5 — random classifier (diagonal line)
- AUC = 1.0 — perfect separation
- AUC = P(model ranks a random positive above a random negative)
- Use **PR-AUC** over ROC-AUC when positives are rare (<5% of data) — ROC artificially flatters due to high TN count

Which metric to use? Medical diagnosis → maximise **Recall** (missing cancer catastrophic). Spam filter → maximise **Precision** (false alarms delete real emails). General → **F1**. Comparing models across thresholds → **AUC-ROC**.

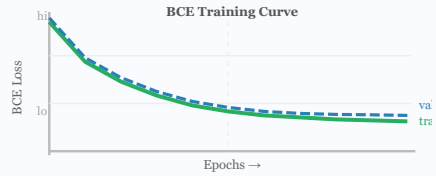
Training Dynamics & the Decision Threshold

⚙ THE DIAL — Decision Threshold τ

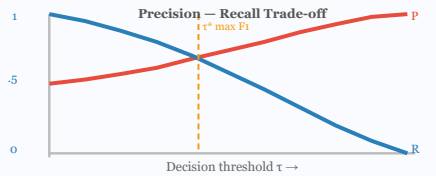
Predict Class 1 when $\hat{y} \geq \tau$. Default $\tau = 0.5$ is rarely optimal. Tune on validation data using the Precision–Recall curve or your domain's cost function.

Setting	Precision	Recall	Use when...
τ high (e.g. 0.8)	↑ Higher	↓ Lower	False alarms are costly
$\tau = 0.5$ (default)	Balanced	Balanced	Starting point only
τ low (e.g. 0.3)	↓ Lower	↑ Higher	Missing cases is costly

Critical: threshold is set *after* training. The model is trained on BCE at its raw output. Threshold is a deployment decision, not a training hyperparameter. One trained model can serve multiple applications with different thresholds.



BCE converges smoothly with Adam. Val tracks train closely — logistic regression rarely overfits severely due to its low parameter count.



Raising τ increases Precision (fewer false alarms) but decreases Recall (more misses). Optimal τ maximises F1.*

BCE Convergence Details

- **Adam (lr=1e−3):** default for logistic regression networks. Adam's per-parameter adaptive LR handles sparse gradient signal at saturation better than plain SGD.
- **LR too high:** BCE oscillates; val loss never converges.
- **Batch size 32–256:** smaller batches → noisier gradients, better generalisation. Larger batches → faster GPU throughput.
- **Convergence:** monitor val BCE; use Early Stopping (Ch.6) to halt when it stalls.

Class imbalance: when one class dominates, pass `class_weight={0:1, 1:ratio}` to `model.fit()`. This scales the loss contribution of the minority class up, preventing the model from collapsing to predict the majority only. Alternative: oversample the minority class (SMOTE).

TensorFlow Implementation — Binary & Multiclass

Architecture: `Dense(1, activation='sigmoid')` + `binary_crossentropy`. Adding `sigmoid` to `Dense(1)` from Ch.1 is the only architectural change required to produce a binary classifier.

```
import tensorflow as tf
import numpy as np

# ── Data: binary classification (linearly separable) ────────
np.random.seed(42)
X = np.random.rand(800, 4).astype(np.float32)   # 4 features
y = (X[:,0] + X[:,1] > 1.1).astype(np.float32)  # linear
boundary

X_train, X_test = X[:640], X[640:]
y_train, y_test = y[:640], y[640:]

# ── Model: Dense(1, 'sigmoid') = logistic regression
────────
model = tf.keras.Sequential([
    tf.keras.layers.Dense(
        units=1,
        activation='sigmoid',   # ← only change from Ch.1
        input_shape=(4,)
    )
])

# ── Compile
──────────────────────────────────────────────────────
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='binary_crossentropy',
    metrics=['accuracy', tf.keras.metrics.AUC(name='auc')]
)

# ── Train
────────────────────────────────────────────────────────
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    verbose=0
)

# ── Evaluate
─────────────────────────────────────────────────────
loss, acc, auc = model.evaluate(X_test, y_test, verbose=0)
print(f"BCE: {loss:.4f}  Acc: {acc:.4f}  AUC: {auc:.4f}")

# ── Custom threshold (set after training)
──────────────────────
probs     = model.predict(X_test, verbose=0).flatten()
threshold = 0.5                # ← tune this on val set
preds     = (probs >= threshold).astype(int)

# ── Multiclass: swap sigmoid → softmax
────────────────────────
# model_mc = tf.keras.Sequential([
#     tf.keras.layers.Dense(n_classes, activation='softmax',
#                           input_shape=(4,))
# ])
# model_mc.compile(optimizer='adam',
#     loss='sparse_categorical_crossentropy',  # y as int
labels
```

Training Loop Steps

1. **Forward:** $z = w \cdot x + b$, then $\hat{y} = \sigma(z)$
2. **Loss:** $BCE = -[y \cdot \log\hat{y} + (1-y) \cdot \log(1-\hat{y})]$
3. **Backward:** $\partial L/\partial z = \hat{y} - y$ — gradient is clean, no vanishing
4. **Update:** $w \leftarrow w - \alpha \cdot \partial L/\partial w$ (Adam adaptive per-weight)
5. **Repeat** until val BCE stalls or Early Stopping fires

Sigmoid vs Softmax

Aspect	Sigmoid	Softmax
Task	Binary / multi-label	Mutually exclusive
Outputs	Independent (0,1) each	Sum to 1 (probability)
Loss	binary_crossentropy	sparse_categorical_CE
Dense	Dense(1)	Dense(n_classes)

🔒 **Key insight:** The output-layer gradient $\partial L/\partial z = \hat{y} - y$ for sigmoid + BCE. The sigmoid derivative and the log cancel algebraically — no vanishing in the final layer. This is the mathematical reason BCE is the correct loss for sigmoid outputs.

Logistic Regression — What Interviewers Actually Ask

Q: Why not use MSE with sigmoid for binary classification?

Two reasons. First, MSE + sigmoid is **non-convex** — the loss surface has local minima and gradient descent is not guaranteed to find the optimal weights. Second, when $\hat{y} \approx 0$ or ≈ 1, $\sigma'(z) \approx 0$. MSE passes this near-zero gradient straight through; weight updates nearly vanish. Binary cross-entropy avoids both: it is convex with sigmoid, and its gradient $\partial L / \partial z = \hat{y} - y$ has no vanishing term — the sigmoid derivative cancels cleanly with the log derivative.

Q: What is the accuracy paradox? When does accuracy fail?

A dataset with 95% class-0. A model that always predicts class 0 achieves 95% accuracy with zero predictive power. Accuracy fails whenever classes are imbalanced *and* you care about the minority class. Alternatives: **F1** (balances P and R), **AUC-ROC** (threshold-independent), **PR-AUC** (preferred when positives are rare, since ROC artificially flatters due to the large TN denominator in FPR).

Q: Is logistic regression a linear classifier?

Yes. The decision boundary is $\mathbf{w}^\top \mathbf{x} + b = 0$ — a hyperplane. The sigmoid only squashes the output; it does not curve the boundary. Any classes that are not linearly separable in the original feature space cannot be correctly classified by logistic regression, regardless of the threshold. To handle non-linear boundaries: add features (e.g. x_1^2), or add a hidden layer with a non-linear activation — which is Ch.3's entire motivation.

Q: What does AUC-ROC actually measure?

AUC = P(model assigns a higher probability to a randomly chosen positive sample than to a randomly chosen negative sample). A model with AUC=0.9 will rank a positive above a negative 90% of the time, regardless of the specific threshold. AUC is scale-invariant and threshold-invariant — useful for model comparison and threshold tuning on separate data. Trade-off: because it is threshold-independent, it does not tell

⚠ Common Interview Traps

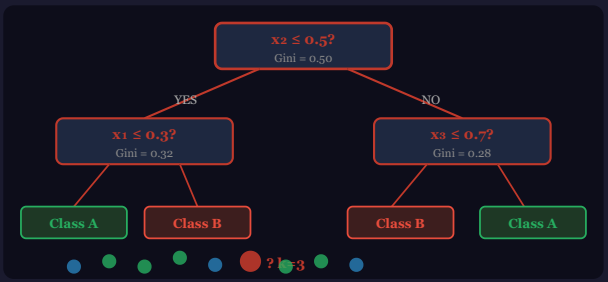
- **Calling logistic regression non-linear:** the decision boundary is linear. The sigmoid is non-linear but only affects the output, not the shape of the boundary itself.
- **Treating τ=0.5 as the correct threshold:** 0.5 is a starting point. Tune on a held-out validation set using the P/R tradeoff curve or a domain-specific cost function.
- **Confusing multi-label with multiclass:** multi-label → multiple positives simultaneously (sigmoid per output). Multiclass → exactly one winner (softmax + sparse CCE).
- **Reporting accuracy on imbalanced data:** always inspect class distribution first. With 95:5 split, AUC and F1 are the meaningful metrics.
- **Forgetting feature scaling:** gradient descent for logistic regression converges faster and more stably when features are standardised. Same constraint as Ch.1.

Assumptions of Logistic Regression

1. **Linear separability:** the optimal boundary is a hyperplane. Violated by XOR — the motivating problem of Ch.3.
2. **No severe multicollinearity:** correlated features destabilise individual weight magnitudes.
3. **Independence of observations:** no autocorrelation between samples. Violated by time series.
4. **No severe class imbalance** without correction: use `class_weight`, threshold tuning, or SMOTE.
5. **Sufficient events per predictor:** rule of thumb — at least 10 positive events per predictor for stable estimates.

🔒 The Bridge Fact Interviewers Test

Logistic regression draws a straight line (hyperplane). XOR requires a *curved* boundary — no linear classifier can solve it regardless of threshold. The only solution is a hidden layer with a non-linear activation. That is the complete motivation for neural networks: not to "be smarter", but to escape the hard constraint of linear decision boundaries.



CHAPTER 10

Classical Classifiers

Decision Trees · Gini & Entropy · K-Nearest Neighbours · When to Use Which

Before neural networks dominated, these three algorithms answered most classification questions. They remain the first baseline any interviewer expects you to reach for — and the benchmark every deep model must beat.

How a Decision Tree Splits

A decision tree recursively partitions feature space using **axis-aligned splits**. At each node it asks: which feature and threshold produces the purest children? Purity is measured by Gini impurity or information entropy.

$$\text{Gini}(S) = 1 - \sum_{k=1}^{K} p_k^2$$

$$H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

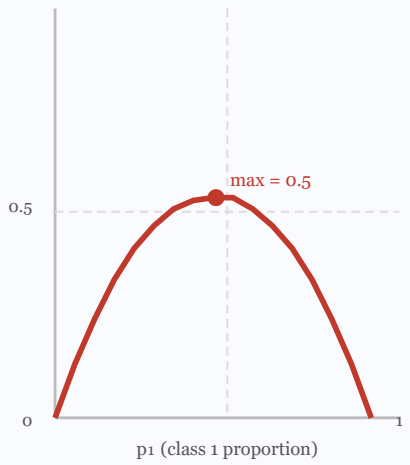
$$\text{IG} = H(\text{parent}) - \sum_j \frac{|S_j|}{|S|} H(S_j)$$

Term Breakdown

- p_k — proportion of class k samples at the node
- **Gini = 0** — perfectly pure node (all one class)
- **Gini = 0.5** — maximally impure (50/50 binary)
- **H = 0** — pure; H = 1 bit — maximally mixed (binary)
- **IG** — Information Gain: entropy reduction after the split
- $|S_j|/|S|$ — weight by fraction of samples going to child j

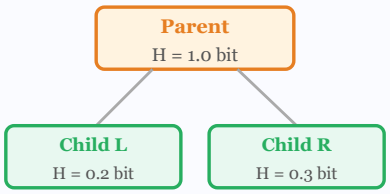
Gini vs Entropy in practice — Gini avoids log computation; marginally faster. Both criteria produce nearly identical trees. sklearn defaults to Gini. The meaningful dial is **max_depth**, not the criterion.

Gini (binary class)



Gini peaks at 0.5 (equal mix); drops to 0 at a pure node.

Information Gain



IG = 1.0 − 0.25 = 0.75 bit

High IG when children are much purer than the parent.

🔒 **The Splitting Algorithm**

The tree evaluates every (feature, threshold) pair at each node. The split with highest IG is chosen. This greedy search never backtracks — it is locally optimal, not globally optimal. A split that looks bad now might enable a better structure below. This is why pruning (max_depth, min_samples_leaf) is critical.

K-Nearest Neighbours

KNN stores the entire training set. At prediction time, it finds the k closest training points by distance and returns the majority class. There is **no training step** — the model *is* the training data.

$$d(\mathbf{x}, \mathbf{x}') = \left(\sum_{i=1}^{n} |x_i - x_i'|^p \right)^{1/p}$$

Minkowski Distance Family

- **p = 1** — Manhattan (sum of absolute differences; robust to outliers)
- **p = 2** — Euclidean (straight-line; sklearn default)
- **p → ∞** — Chebyshev (max over dimensions)
- **Text / sparse data** — cosine similarity instead (angle, not magnitude)

Always scale features before KNN. A feature with range [0, 10,000] contributes 10,000× more to Euclidean distance than one with range [0, 1]. Use `StandardScaler` or `MinMaxScaler` before fitting.

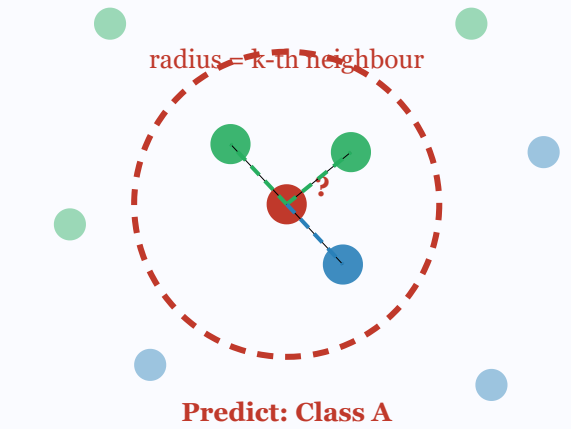
Curse of Dimensionality

In high dimensions, all pairwise distances converge to the same value. With 100+ features, "nearest neighbours" are no longer meaningfully close. Apply PCA or feature selection before KNN on high-dimensional data.

Complexity

- **Train:** O(1) — store the dataset
- **Predict:** O(n · d) per query — scan all n points across d features
- **KD-tree / Ball-tree:** O(log n) for low-d; degrades in high dimensions

KNN: k = 3



k=3: 2 votes Class A, 1 vote Class B → predicts A.

Weighted KNN — Instead of majority vote, weight each neighbour's contribution by 1/distance. Closer neighbours dominate. Set `weights='distance'` in sklearn. Useful when k must be large to filter noise.

Hyperparameter Dials & When to Use Which

Decision Tree Dials

Dial	Too High	Too Low	Start
max_depth	Memorises noise	Underfit	3–8
min_samples_split	Underfit	Overfit	2–20
min_samples_leaf	Underfit	Overfit	1–10
criterion	gini ≈ entropy; gini faster		gini

KNN Dials

Dial	Too High	Too Low	Start
k	High bias, smooth boundary	Memorises noise	√n
metric	euclidean default; cosine for text		euclidean
weights	'uniform' or 'distance'		uniform

DT vs Logistic Regression vs KNN — Selection Guide

Criterion	Decision Tree	Logistic Regression	KNN
Decision boundary	Axis-aligned rectangles	Linear hyperplane	Non-linear, local
Feature scaling needed?	No	Yes (regularisation)	Yes (distance)
Handles non-linearity?	Yes (depth ≥ 2)	No (linear only)	Yes (local)
Probabilistic output?	Rough (leaf freq.)	Yes (calibrated)	Rough (vote frac.)
Interpretability	High (tree path)	High (coefficients)	None
Scales to large n?	Yes	Yes	No (O(nd) predict)
Scales to large d?	Degrades on irrelevant features	Regularisation helps	Fails (curse of dim.)
Missing values?	Handles natively	Requires imputation	Requires imputation
Best for	Mixed types, interactions, explainability	Calibrated probs, linear boundary	Small datasets, local pattern baseline

Default Protocol: Start with Logistic Regression as baseline. Large accuracy gap vs DT → non-linear structure. KNN beats LR on small data → local structure dominates. Both fall short → escalate to XGBoost (Ch.11).

sklearn Implementation

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
import numpy as np

# ── Decision Tree ──────────────────────────────
dt = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,          # dial: controls overfitting
    min_samples_leaf=4,   # dial: minimum node size
    random_state=42
)
dt.fit(X_train, y_train)

# Feature importance (Gini reduction per feature)
importances = dt.feature_importances_   # use for
feature selection

# ── KNN (always scale first!) ──────────────────
scaler = StandardScaler()
X_tr_sc = scaler.fit_transform(X_train)
X_te_sc = scaler.transform(X_test)      # transform,
NOT fit_transform

knn = KNeighborsClassifier(
    n_neighbors=7,        # dial: try sqrt(n_train)
    metric='euclidean',
    weights='distance'    # closer neighbours weighted
more
)
knn.fit(X_tr_sc, y_train)

# ── Logistic Regression (baseline) ─────────────
lr = LogisticRegression(
    C=1.0,                # dial: inverse
regularisation strength
    max_iter=1000,
    random_state=42
)
lr.fit(X_tr_sc, y_train)

# ── Compare via cross-validation ───────────────
for name, mdl, X in [('DT', dt, X_train),
                     ('KNN', knn, X_tr_sc),
                     ('LR', lr, X_tr_sc)]:
    s = cross_val_score(mdl, X, y_train, cv=5,
scoring='f1')
    print(f"{name}: {s.mean():.3f} ± {s.std():.3f}")
```

Ch.10 — Core Interview Checklist

Decision Trees

- Gini vs entropy: near-identical
- max_depth: primary dial
- Axis-aligned splits only
- Greedy, not globally optimal
- No feature scaling needed

KNN

- Always scale features first
- k = √n starting point
- O(nd) predict cost
- Curse of dimensionality
- No training step

Selection

- LR first: linear baseline
- DT: interactions, mixed types
- KNN: small n, local patterns
- High-d: reduce dims first
- Escalate to XGBoost (Ch.11)

Finding optimal k: Loop k from 1 to √(2n), plot cross-val F1. Pick the elbow. Use odd k for binary classification to avoid ties.

DT feature importance: `dt.feature_importances_` returns total Gini reduction per feature. Use this to select features before feeding into more expensive models.

Ch.10 established the three baseline classifiers — Decision Trees partition space into axis-aligned rectangles; KNN votes by distance; Logistic Regression finds one linear boundary. Ch.11 introduces ***Support Vector Machines****, which find the maximum-margin linear boundary, and* ***XGBoost****, which stacks decision trees sequentially so each tree corrects its predecessor's errors. The connecting concept:* ***margin maximisation and boosting vs bagging****.*

Interview Q&A

Gini vs entropy — which criterion should you choose and why?

In practice, both produce nearly identical trees. Gini avoids log computation and is marginally faster. Entropy is slightly more informative when splits are very unequal. sklearn defaults to Gini. The meaningful choice is `max_depth`, not the criterion — and cross-validation will call the bluff either way.

Why does a decision tree overfit and how do you fix it?

An unconstrained tree grows until every leaf is pure, memorising the training set. Fix: set `max_depth` (3–8), `min_samples_leaf` (≥4), or `min_samples_split`. Better still: use Random Forest (bagging, reduces variance) or XGBoost (boosting, reduces bias sequentially). Both use decision trees as the base learner.

Why must you scale features before KNN?

KNN computes Euclidean distance. A feature with range [0, 10,000] contributes 10,000× more to distance than one with range [0, 1] — it completely dominates the neighbourhood. `StandardScaler` (zero mean, unit variance) makes every feature contribute proportionally. Always fit the scaler on training data only, then transform both train and test.

Decision tree vs logistic regression — when does each win?

LR wins when the boundary is linear, when you need calibrated probabilities, or when features are continuous and well-scaled. DT wins when features interact (XOR-like structure), when data includes mixed types or missing values, or when you need a visual explanation that non-technical stakeholders can follow. Cross-validate both; the accuracy gap reveals the boundary shape.

What happens to KNN as k → n?

All training points vote, so the model predicts the majority class everywhere — maximum bias, zero variance, useless classifier. As k → 1, the model memorises training data: 0% training error, high variance. The optimal k lives between these extremes and is found via cross-validation. Use odd k for binary classification to break ties.

Common Traps

- Using KNN without feature scaling — Euclidean distance is meaningless on unscaled features
- Reporting DT training accuracy — unpruned trees approach 100% training accuracy trivially; always report test or CV score
- Choosing k=1 because training accuracy is highest — it will fail on held-out data
- Comparing LR and DT on accuracy alone without checking class imbalance or calibration
- Applying DT to raw image pixels or raw text without feature engineering — axis-aligned pixel splits carry no semantic meaning

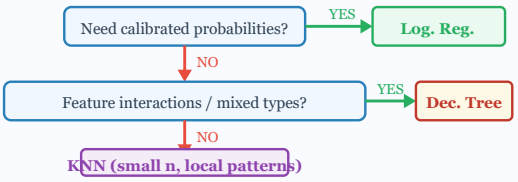
Assumptions

- **Decision Tree:** no distributional assumptions; axis-aligned splits only (no diagonal boundaries)
- **KNN:** local homogeneity — nearby points in feature space share the same label
- **KNN:** distance is meaningful — features must be on comparable scales
- **All three:** features are independent scalars in a flat vector; no spatial or temporal structure

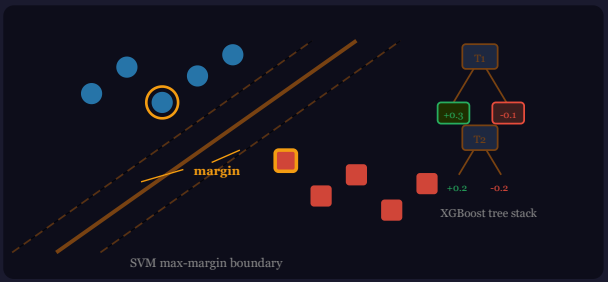
🔒 Bridge Fact

Decision trees are the building block of both Random Forest (bagging: train N trees independently on bootstrapped samples, average predictions — reduces variance) and XGBoost (boosting: train trees sequentially, each correcting the residual errors of the previous — reduces bias). Same weak learner; two opposite aggregation strategies. This distinction is the most common ensemble interview question.

Classifier Selection — Quick Flow



Quick selection flow — cross-validate all three; escalate to XGBoost when none suffice.



CHAPTER 11

SVM & Ensembles

Maximum Margin · Kernel Trick · C & ε · Bagging vs Boosting · XGBoost

SVM finds the widest street between classes. XGBoost builds a series of trees where each one corrects what the last got wrong. Together they cover the two dominant strategies for squeezing accuracy out of structured data.

Support Vector Machines

SVM finds the hyperplane that **maximises the margin** — the gap between the two class boundaries. Only the training points on the margin boundary (support vectors) define the hyperplane. All other points are irrelevant once training is complete.

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$$

$$\text{Soft margin: } \min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i$$

Term Breakdown

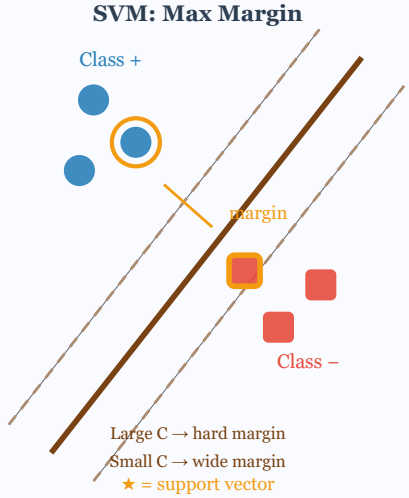
- $\|\mathbf{w}\|^2$ — inverse margin width; minimising = maximising margin
- $y_i \in \{-1, +1\}$ — class labels; constraint requires correct side of margin
- $\xi_i \geq 0$ — slack variables; allow misclassification at a cost
- **C** — regularisation: large C = hard margin (overfit risk); small C = wide margin, tolerates more errors
- **Support vectors** — only points on the margin boundary ($y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$)

SVM for Regression (ε-SVR)

- Predicts a continuous value; ignores errors smaller than **ε**
- Only points *outside* the ε-tube become support vectors
- Larger ε = fewer support vectors = simpler model
- `SVR(kernel='rbf', C=1.0, epsilon=0.1)` in sklearn

> Always standardise features before SVM. The margin computation depends on Euclidean distances — unscaled features break it the same way they break KNN.

SVM: Max Margin



Support vectors (circled) sit on the margin. Maximising margin = minimising $\|\mathbf{w}\|$.

The Kernel Trick

Non-linearly separable data in the original space may become linearly separable in a higher-dimensional space. The kernel trick computes the inner product in that high-d space *without explicitly mapping there.*

$$K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$$

Kernel	Formula	When to use
Linear	$\mathbf{x}^\top \mathbf{x}'$	Linearly separable; large n
RBF / Gaussian	$\exp(-\gamma \lVert \mathbf{x} - \mathbf{x}' \rVert^2)$	Default; unknown structure
Polynomial	$(\mathbf{x}^\top \mathbf{x}' + c)^d$	Structured feature interactions

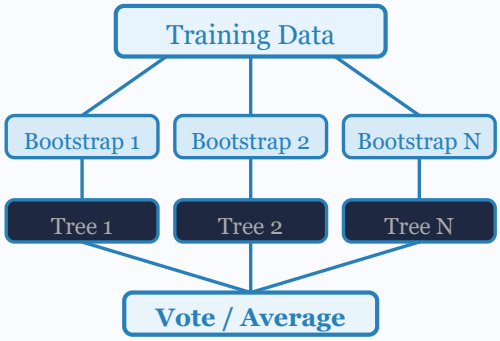
RBF Dial: γ

- **Large γ** — narrow Gaussian; each point influences only nearby region → overfit, wiggly boundary
- **Small γ** — wide Gaussian; smoother boundary; may underfit
- **Start:** `gamma='scale'` (1 / (n_features × X.var()))

> SVM with RBF kernel is *O(n²)* to train and *O(n_sv · d)* to predict (n_sv = number of support vectors). For n > 10 000, use `LinearSVC` or kernel approximations (`Nystroem`).

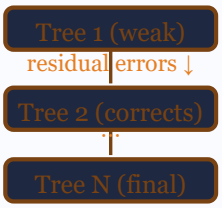
Bagging vs Boosting

Bagging (Random Forest)



↓ Variance

Boosting (XGBoost)



↓ Bias

Bagging: parallel, independent trees reduce variance. Boosting: sequential trees reduce bias.

XGBoost — How It Works & What to Tune

XGBoost fits shallow trees sequentially. Each tree fits the **negative gradient of the loss** (pseudo-residuals) from all previous trees. The final prediction is the sum of all tree outputs scaled by the learning rate.

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \cdot f_t(\mathbf{x}) \qquad \text{where } f_t \text{ minimises } \sum_i l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t)$$

Core Dials

Dial	Too High	Too Low	Start
n_estimators	Overfit (slow)	Underfit	100–1000
learning_rate (η)	Overfit quickly	Needs more trees	0.05–0.3
max_depth	Overfit, slow	Underfits	3–6
subsample	Low variance reduction	Underfit	0.6–0.9
colsample_bytree	Low diversity	Underfit	0.6–0.9
reg_alpha (L1)	Too sparse	No reg	0–1
reg_lambda (L2)	Underfit	No reg	1 (default)

Tuning order: Set n_estimators high. Tune max_depth + min_child_weight. Then learning_rate + n_estimators together. Then subsample + colsample_bytree. Finally regularisation.

XGBoost vs Random Forest

Property	XGBoost	Random Forest
Training	Sequential (boosting)	Parallel (bagging)
What it reduces	Bias	Variance
Typically more accurate?	Yes (on tabular)	Competitive baseline
Overfit risk	Higher (more dials)	Lower
Missing values	Handles natively	Needs imputation
Built-in regularisation	L1 + L2	max_features only
Speed	Slower to tune	Faster, fewer dials
Interpretability	feature_importances_	feature_importances_

When to start with Random Forest: Quick baseline; noisy data; minimal tuning time. Switch to XGBoost when you need that extra 1–3% and can afford a hyperparameter search.

🔒 **The η × n_estimators Trade-off**
Lower η always improves generalisation — but requires proportionally more trees. Set early stopping (`early_stopping_rounds=50`) and train with a low η (0.01–0.05). Let the number of trees be determined by when val loss stops improving.

sklearn & XGBoost Implementation

```
from sklearn.svm import SVC, SVR
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb

# ── SVM (always scale!) ──────────────────────────
scaler = StandardScaler()
X_tr_sc = scaler.fit_transform(X_train)
X_te_sc = scaler.transform(X_test)

svm = SVC(
    kernel='rbf',           # 'linear', 'poly', 'rbf'
    C=1.0,                  # dial: soft margin penalty
    gamma='scale',          # dial: RBF width
    probability=True        # enables predict_proba
(slower)
)
svm.fit(X_tr_sc, y_train)
# n_support_: inspect support vector counts per class

# ── Random Forest (baseline ensemble) ────────────
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,         # fully grown trees (pruned
by min_samples)
    max_features='sqrt',    # feature subsampling per
split
    random_state=42
)
rf.fit(X_train, y_train)

# ── XGBoost ──────────────────────────────────────
model = xgb.XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,     # dial: low LR + many trees
    max_depth=4,            # dial: shallow trees reduce
overfit
    subsample=0.8,          # stochastic boosting
    colsample_bytree=0.8,
    reg_lambda=1.0,         # L2 regularisation
    early_stopping_rounds=50,
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    verbose=False
)
print("Best iteration:", model.best_iteration)
```

Ch.11 — Core Interview Checklist

SVM

- Max-margin hyperplane
- Support vectors only
- C: margin vs error trade-off
- Kernel trick: implicit high-d
- RBF γ: width of influence

Bagging / RF

- Bootstrap + parallel trees
- Reduces variance
- max_features = √d per split
- OOB error estimate
- Robust starting point

XGBoost

- Sequential, corrects residuals
- Reduces bias
- η low + n_estimators high
- Early stopping
- Handles missing values

SVM vs XGBoost on tabular data: SVM scales poorly ($O(n^2)$ training). XGBoost is usually preferred for $n > 10{,}000$. SVM excels in text classification and high-dimensional sparse data (linear kernel).

Out-of-bag (OOB) error in Random Forest is a free cross-validation estimate. Each tree is evaluated on the ~37% of training samples not in its bootstrap. Set `oob_score=True`.

Ch.11 established maximum-margin classification (SVM), variance-reducing ensembles (Random Forest), and bias-reducing gradient boosting (XGBoost) — all requiring labelled data. Ch.12 removes that requirement: ***clustering*** *discovers structure and group memberships with no labels at all. The connecting concept:* ***how to evaluate a model when there are no ground-truth labels to compare against****.*

Interview Q&A

What is the kernel trick and why does it matter?

Non-linearly separable data may become linearly separable in a higher-dimensional space. The kernel trick computes the inner product in that space via $K(x,x') = \varphi(x)\cdot\varphi(x')$ — without ever explicitly computing $\varphi(x)$. The RBF kernel corresponds to an infinite-dimensional feature space. No explicit mapping = no storage cost.

What does C control in SVM and what happens at extremes?

C is the penalty per misclassified training point. Large C: the model tries hard to classify everything correctly → narrow margin, high variance, overfit risk. Small C: tolerates errors → wide margin, simpler boundary, possibly underfit. C is the inverse of L2 regularisation strength.

Bagging vs boosting — which reduces variance and which reduces bias?

Bagging trains trees independently on bootstrap samples and averages. Independent trees have uncorrelated errors; averaging reduces variance. Boosting trains trees sequentially; each tree fits the residual errors of the ensemble so far, targeting the remaining bias. XGBoost adds L1/L2 regularisation on top of gradient boosting to prevent overfit.

Why does XGBoost outperform Random Forest in practice?

Boosting corrects errors sequentially → consistently lower bias. XGBoost adds built-in L1/L2 regularisation, handles missing values natively (learns best default direction at each split), uses second-order gradients (Newton boosting) for faster convergence, and histogram binning for speed. Random Forest is more robust to noise and requires less tuning.

What is ε-insensitive loss in SVR?

SVR ignores prediction errors smaller than ε. Points within the ε-tube contribute nothing to the loss or to the model. Only points outside the tube (support vectors) define the regression function. Larger ε = fewer support vectors = simpler, smoother model. C still controls how harshly violations beyond ε are penalised.

Common Traps

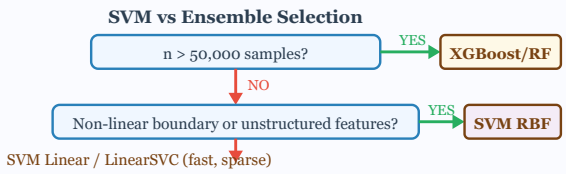
- Forgetting to scale before SVM — RBF distance is just as sensitive as Euclidean distance
- Tuning n_estimators first in XGBoost — tune depth and LR first, then use early stopping to find n_estimators
- Confusing bagging (variance reduction) with boosting (bias reduction) in interview explanations
- Using SVM with RBF kernel on $n > 100{,}000$ without approximation — $O(n^2)$ training time is prohibitive
- Setting learning_rate high to train faster — reduces generalisation; always combine low η with early stopping

Assumptions

- **SVM:** features on comparable scale (standardise); optimal C and γ are data-dependent
- **SVM:** solution is a convex optimisation — guaranteed global optimum unlike neural networks
- **Random Forest:** trees should be diverse; too-high correlation between trees reduces the variance reduction benefit
- **XGBoost:** differentiable loss function required; works with any twice-differentiable objective

🔒 Bridge Fact

SVM and XGBoost both require labelled data. The next frontier is learning from *unlabelled* data — the majority of data in most real-world settings. Ch.12 covers K-Means, DBSCAN, and HDBSCAN: algorithms that find natural groupings with no labels at all, and how those groupings can be used as features in downstream supervised models.



Model selection flow for SVM and ensemble methods.